Course Overview

COSC 1210, Computer Science I, Augustana U

Table of Contents

- 1. Objectives
- 2. Get to know your teacher
- 3. Understand what this course is about
- 4. Find out where to find stuff
- 5. Find out whom to ask when
- 6. Find out about the formal stuff
- 7. Understand how you're being graded
- 8. Find out what you'll do every week
- 9. Setup: What you'll need and what I use
- 10. Get your next assignments
- 11. Find out what's next
- 12. Glossary

1. Objectives

- Get to know your teacher
- Share why you are here
- Check out the entry survey results
- Understand what this course is about
- Find out where to find stuff
- Find out when to ask
- Find out about the formal stuff
- Understand how you're being graded
- Find out what you'll do every week
- Get your next assignments
- Find out what's next

2. Get to know your teacher

- For the professional stuff, I asked Gemini: https://share.gemini.google/lbAsXSSlJgdJ

  > I am going to take a course with Marcus Birkenkrahe, Computer Science I (introduction to programming with Python). Tell me something about him that might be interesting and useful.

- Privately:
  - PhD in theoretical particle physics (lattice gauge theory)
  - Worked at large multi-national companies in IT positions
  - Came to the US in 2021 from Germany (US citizen since 2026)
  - Family: 1 wife, 1 daughter, 2 grandchildren, 3 dogs, 2 cats
  - Favorite Operating System (OS): Linux
  - Favorite programming language: C
  - Favorite programming environment: Emacs (Lisp)
  - Favorite food: Schnitzel (aka Milanesa, tonkatsu, escalope)
  - Favorite drink: peppermint tea

PRACTICE Your turn - why are you here?

- Even if you filled in the entry survey:
- Let us hear why you're here - One-word check-in
- So that I can hear (and learn) your name.

3. Understand what this course is about

- This is what the catalog says:

  > "An introduction to **computer science**, which include topics such as **software engineering**, computer **architecture**, and programming **languages**. Emphasis on learning the **styles**, **techniques**, and **methodologies** necessary to **design** and **develop readable and efficient** programs."

- What this means:

  > CS is a house with many rooms, many of them under construction. In this course you only get to dip your toe in the water by learning something about it. But **how you do anything is how you do everything**, so: You should approach CS as a professional from the start - no matter what your calling is, e.g. another STEM field, business, arts, or whatever.

- This is what the course is really about:

  1. Learn how to tell a machine what to do.
  2. Learn how to write programs that work.
  3. Learn to use the computer as a tool.

4. Find out where to find stuff

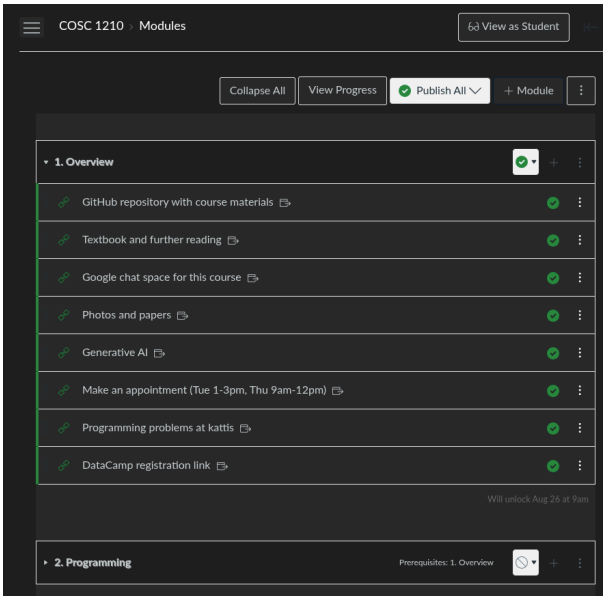
- Canvas
- GitHub

- Textbook
- DataCamp
- Google Space
- YouTube Playlist
- Photos
- Chatbots / Generative AI

PRACTICE From Canvas to the textbook

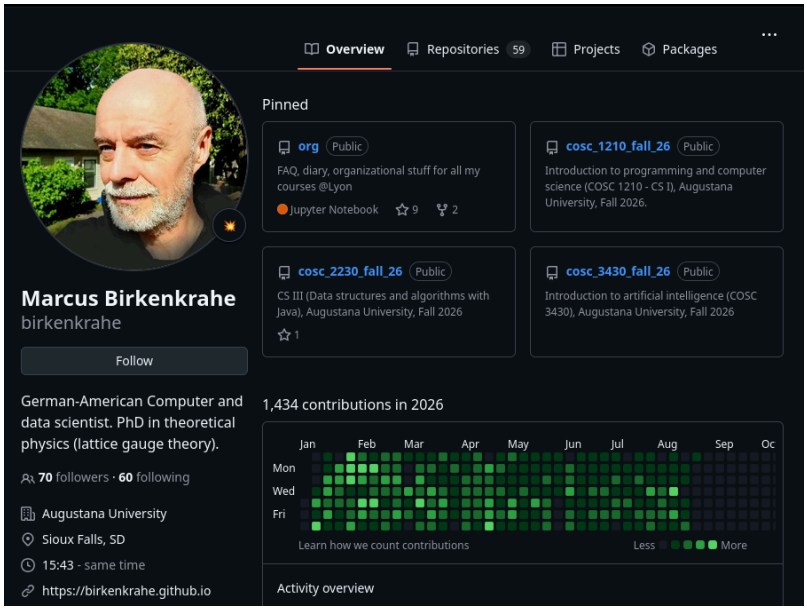
1. Open the Canvas course (https://augie.instructure.com/courses/15635)
2. Open the first module to get to the links
3. Follow along

Canvas



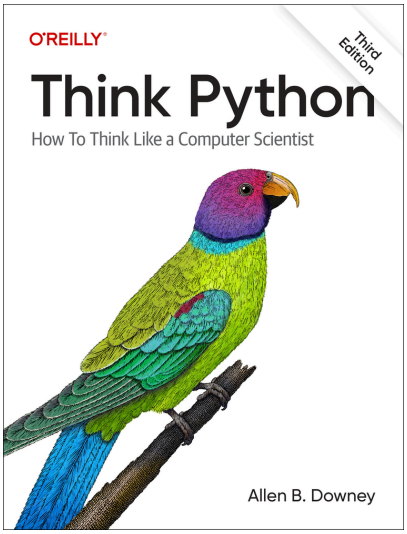
- A learning management system (LMS)
- This is a special case of a content management system (CMS)
- We use Canvas for:
    1. Announcements
    2. Assignments and Quizzes
    3. Pages for course information
    4. Grades (as up to date as I can manage)
    5. Google Meet (link - I record all of my sessions)
    6. Google Drive (link - session recordings, whiteboard screenshots)
    7. YuJa (occasional videos)
    8. Syllabus (course overview, grading, schedule)
    9. Attendance (transfer from Google Forms)
    10. Respondus Lockdown Browser (might use it for tests)

GitHub



- Do you have a GitHub profile? Have you used Git before?
- GitHub (and other platforms) use the Git version control system
- If not, register for free and complete the Hello World project
- The course materials are here: https://github.com/birkenkrahe/cosc_1210_fall_26
- All lectures, assignments, tests are in the `org/` directory
- These are Emacs Org-mode files (`.org`) rendered as Markdown (`.md`)
- There will also be PDFs, data, source files, and images
- I use GitHub to merge files from different devices into one repo

Textbook



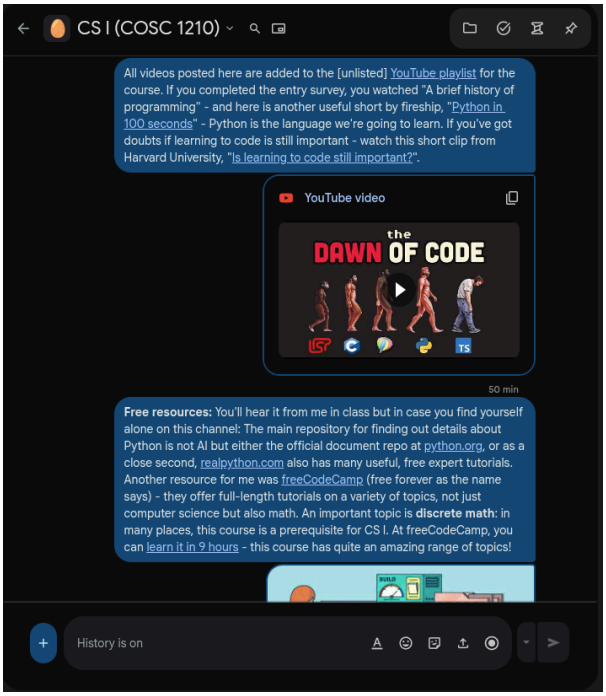
- We are going to use the textbook "Think Python" by Allen B. Downey, 3rd edition - free and online: https://allendowney.github.io/ThinkPython/

- Readings from the book are aligned with our progress.
- The booksite contains the entire book and `Colab` notebooks where you can read through the book and interactively run all of the code.

DataCamp, Kattis, and other platforms

- DataCamp is a top data science learning platform. You have access to 600+ courses for free.
- I am offering this as a practice ground, and I will occasionally refer to a DataCamp course, tutorial or project in class.
- If you've got more energy and enthusiasm after doing your chores, then DataCamp may be a fun activity.
- You find the registration link in Canvas. Must register with DataCamp.com first.
- Kattis is a site with problems for competitive programming in many different languages including Python. Start with the hello world problem to see how it works and then keep solving problems.

Google Chat Space and YouTube Playlist



- I do a fair amount of "work on the side": reading, coding, etc.
- In order to protect your precious email space, I'll use Google space.
- I encourage you to use that space to discuss, share, comment.
- This space will stay live across multiple courses after the course.
- Feel free to join or leave it at any time.
- You'll be invited to the space by me.
- Link to the YouTube playlist: youtube.com/watch?v=9uW6B9LPntY&list=PLRe9Qsi9YpYU

Photos (GDrive)

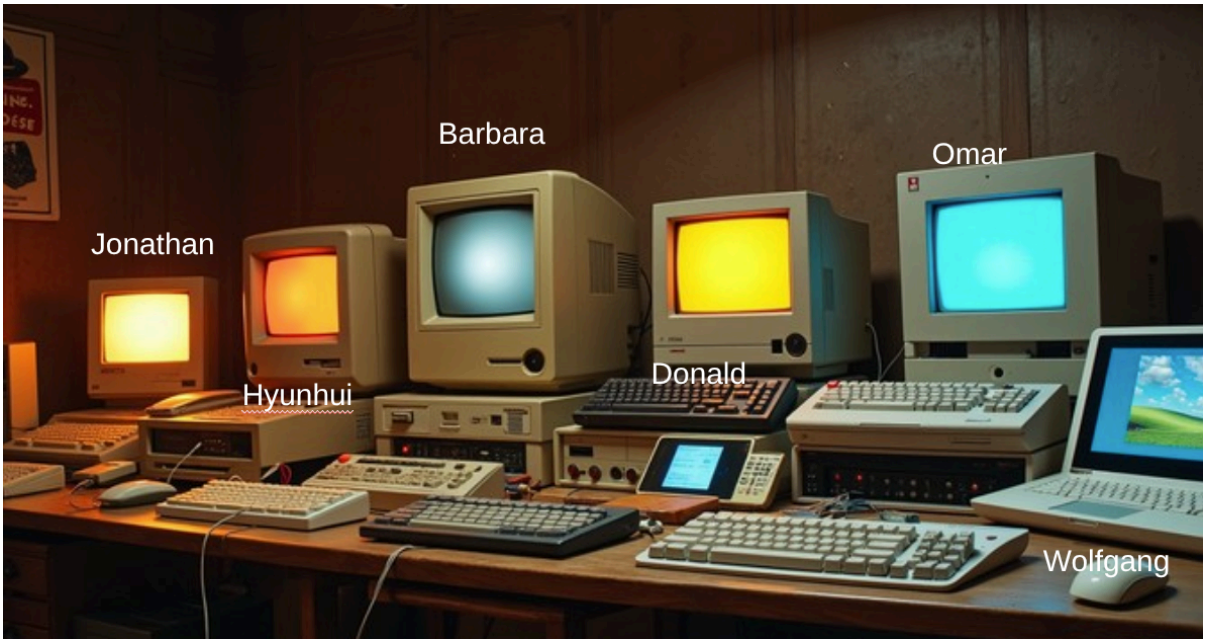


Figure 1: I will ask you to put your names into the class photo

- I upload photos to GDrive: tinyurl.com/photos-fa26-1210
- The class photo (to help me learn your names) is also here as an editable Powerpoint slide: **please add your name** to your image.

Chatbots / Generative AI Assistants

- Gemini with Google Colab is a true agent + notebook solution that you can use to write Python programs online for free (within set limits). For heavy use, you'll soon run out of tokens, however.

- Privately (since 05/26), I use Claude Code (agent harness using Claude). If you're strapped for cash, I recommend the free combo: OpenCode (harness) + Ollama (server) + Qwen3-Coder (model).
- I use generative AI to:
  - Drill me before lectures (as a tutor)
  - Generate quizzes from different sources
  - Give me ideas for new exercises
  - Check my lecture scripts against the textbook
- I have published about teaching computer and data science with and without AI (see birkenkrahe.github.io or researchgate.net)

PRACTICE How do you use AI?

- Clean up my notes and give me a summary
- Tell me how long I should study for assignment X
- Debug my code after trying other resources
- Sort through large amounts of data give me patterns (shopping)
- Manage my bank transaction for budgeting purposes
- Tell me a problem (at my level) and don't give me the answer

5. Find out whom to ask when

- Always ask your mates/peers/friends first
- If you don't know anyone, make contact with someone
- Next, ask me before, in, or after class
- Come to my office hours: Tue/Thu 9-10 am and 2-3 pm

6. Find out about the formal stuff

- Read the fine print in the syllabus.
- This course is an **AI-enabled course:** AI tools are welcome for research, debugging, and drafting, but every line of code you submit must be code that you understand and can explain live; disclosed use is fine,

undisclosed or unexamined AI-written programs are not. More details in the syllabus.
- Feel free to inform me when I make mistakes.

7. Understand how you're being graded

What	When	How much
Programming assignments	weekly	40%
Quizzes (online)	weekly	25%
Participation	always	10%
Midterm exam	TBD	15%
Final exam (optional)	TBD	10%
Total		100%

- Programming assignments: weekly, submit via URL
- There will be plenty of (optional) bonus challenges
- Quizzes: multiple choice, not timed, home, individual
- Participation: present solutions, ask/answer questions, upload complete notebooks from live-coding sessions (code along)
- Midterm exam: 50 min, individual, based on quizzes (with mock exam)
- Final exam: 30 min, optional, individual conversation

8. Find out what you'll do every week

1. Overview (theory)
2. Example (code along)
3. Practice (your turn)
4. Problem (home assignment)
5. Review (following class)

Detailed breakdown

In class:

Retrieval warm-up	**5 min**
Lecture/demo	**20 min**
Live coding	**20 min**
Applied practice	**15 min**
Debrief	**5 min**
Summary	**5 min**
Total	70 min

Exams:

Popquiz	10 min	Surprise
Midterm	70 min	TBD
(Optional oral) Final	30 min	TBD

At home (survival minimum):

Quiz	**60 min**	**Weekly**
Assignment	**60 min**	**Weekly**
Reading	**60 min**	**Weekly**
Review	**60 min**	**Weekly**
Total	**4 hrs**	Weekly

[Usual for a 3 credit class: 3 * 2-3 hrs = 6-9 hrs]

9. Setup: What you'll need and what I use

- **Live-coding and assignments**: We'll mostly work in JupyterLite - a full Jupyter notebook that runs entirely in your browser. No installation, no account, nothing to configure - open the link and start coding in Python.
- Something even lighter than JupyterLite, with no notebook cells? Try onecompiler.com/python - a plain online Python editor and runner.
- Want to run Python on your own machine too? Install it from python.org/downloads. It's free, and installers exist for Windows, macOS, and Linux.

- What else do you need?

  > **An editor or IDE** - your choice, entirely, if/when you work locally. VS Code, Notepad++, vim, Emacs, whatever you're comfortable with or want to learn. I won't teach or require any particular one.

10. Get your next assignments

- First **quiz** based on all materials from this week (due Fri 9/4 2pm)
- Simple **home assignment** with Jupyter notebooks (due Fri 9/4 2 pm)
- Upload code-along notebook (due Fri 9/4 2 pm) for participation
- Optional: register with DataCamp and take a look
- Reading: chapter 1, Think Python.

11. Find out what's next

- Programming as a way of thinking (ch 1)
- Code-along with me: Getting started with Jupyter

12. Glossary

Course platforms & tools

Term	Definition
Canvas	LMS: announcements, assignments, grades
GitHub	Hosts course materials via Git
DataCamp	Free platform, 600+ coding courses
Kattis	Practice problems at open.kattis.com
Google Meet	Video platform; sessions recorded here
Google Drive	Stores recordings, whiteboard photos
Google Chat Space	Class discussion and Q&A space
YouTube	Course-related video playlist
YuJa	Occasional supplementary video hosting
Respondus Lockdown Browser	Locked browser, possibly for tests

AI tools

Term	Definition
Gemini	Google's chatbot/agent used for research demos
Google Colab	Free notebook environment; runs Gemini agent
Claude Code	Agent harness using Claude (instructor's tool)
OpenCode	Free/open agent harness, alt. to Claude Code
Ollama	Runs open LLMs locally, for free
Qwen3-Coder	Free open coding model, pairs with Ollama

Setup & dev tools

Term	Definition
JupyterLite	Browser-based Jupyter notebook, no install needed
python.org/downloads	Official installer for local Python
onecompiler.com/python	Plain online Python editor and runner
Emacs	Extensible text editor; author's tool
Org-mode	Emacs mode for notes, code, literate docs

Author: Marcus Birkenkrahe
Created: 2026-09-07 Mon 20:45